Quick Sort

Quick Sort is a divide and conquer algorithm. It creates two empty arrays to hold elements less than the pivot value and elements greater than the pivot value, and then recursively sort the sub arrays.

It is also called **partition-exchange sort**. This algorithm divides the list into three main parts:

1. Elements less than the **Pivot** element
2. Pivot element(Central element)
3. Elements greater than the pivot element

Pivot element can be any element from the array, it can be the first element, the last element or any random element. In this tutorial, we will take the rightmost element or the last element as **pivot**.

For example: In the array `{52, 37, 63, 14, 17, 8, 6, 25}`, we take `25` as **pivot**. So after the first pass, the list will be changed like this.

`{6 8 17 14 25 63 37 52}`

Hence after the first pass, pivot will be set at its position, with all the elements **smaller** to it on its left and all the elements **larger** than to its right. Now `6 8 17 14` and `63 37 52` are considered as two separate sunarrays, and same recursive logic will be applied on them, and we will keep doing this until the complete array is sorted.

Quick Sort Algorithm: Steps on how it works:

1. Find a “pivot” item in the array. This item is the basis for comparison for a single round.
2. Start a pointer (the left pointer) at the first item in the array.
3. Start a pointer (the right pointer) at the last item in the array.
4. While the value at the left pointer in the array is less than the pivot value, move the left pointer to the right (add 1). Continue until the value at the left pointer is greater than or equal to the pivot value.
5. While the value at the right pointer in the array is greater than the pivot value, move the right pointer to the left (subtract 1). Continue until the value at the right pointer is less than or equal to the pivot value.
6. If the left pointer is less than or equal to the right pointer, then swap the values at these locations in the array.
7. Move the left pointer to the right by one and the right pointer to the left by one.
8. If the left pointer and right pointer don’t meet, go to step 1.

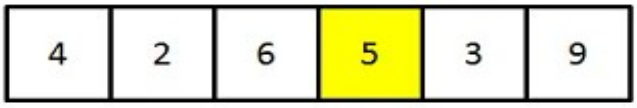
Important Characteristics of Quick Sort:

- Quick Sort is useful for sorting arrays.
- In efficient implementations Quick Sort is not a stable sort, meaning that the relative order of equal sort items is not preserved.
- Overall time complexity of Quick Sort is O(nLogn). In the worst case, it makes O(n^2) comparisons, though this behavior is rare.
- The space complexity of Quick Sort is O(nLogn). It is an in-place sort (i.e. it doesn’t require any extra storage)

Example:

Let's consider an array with values `{4, 2, 6, 5, 3, 9}`

Step 1
Determine pivot



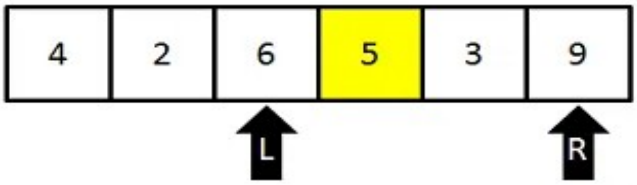
Step 2
Start pointers at left and right



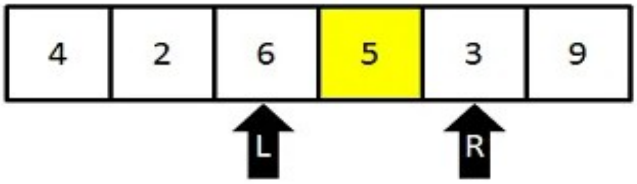
Step 3
Since 4 < 5, shift left pointer



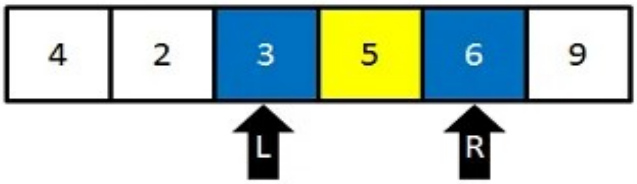
Step 4
Since 2 < 5, shift left pointer
Since 6 > 5, stop



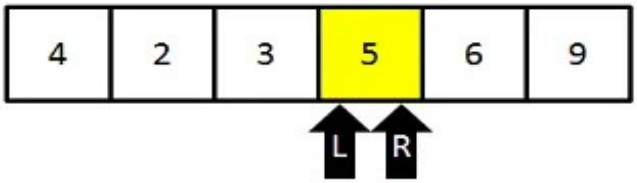
Step 5
Since 9 > 5, shift right pointer
Since 3 < 5, stop



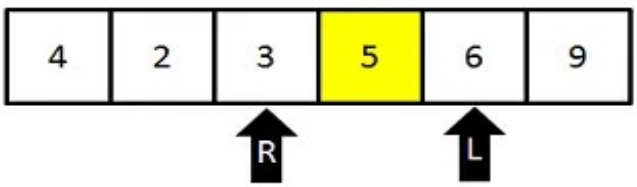
Step 6
Swap values at pointers



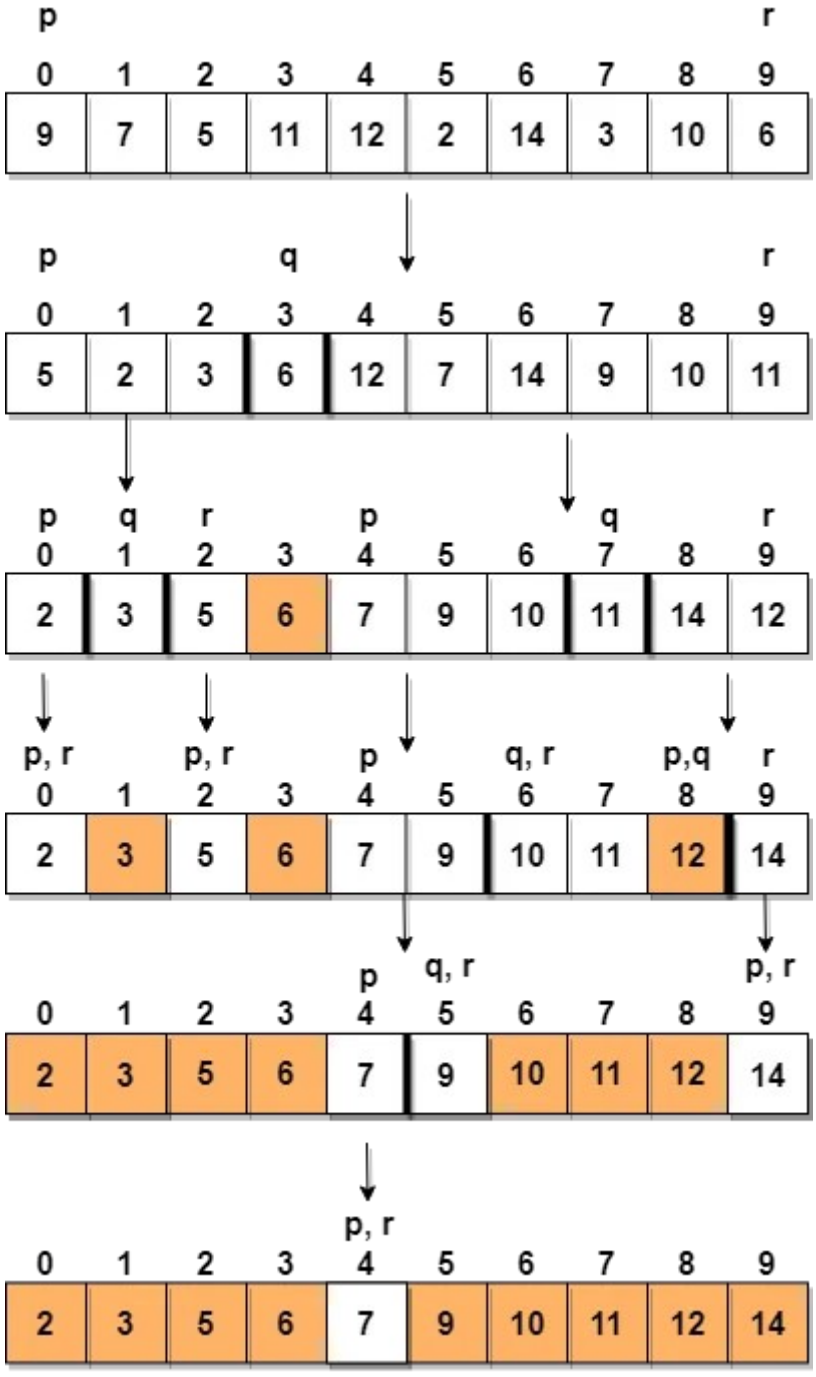
Step 7
Move pointers one more step



Step 8
Since 5 == 5,
move pointers one more step
Stop



2. Let's consider an array with values `{9, 7, 5, 11, 12, 2, 14, 3, 10, 6}`



// simple C program for Quick Sort

```
#include <stdio.h>
int partition(int a[], int beg, int end);
void quickSort(int a[], int beg, int end);
void main()
{
   int i;
   int arr[10]={90,23,101,45,65,28,67,89,34,29};
   quickSort(arr, 0, 9);
   printf("\n The sorted array is: \n");
   for(i=0;i<10;i++)
   printf(" %d\t", arr[i]);
}
int partition(int a[], int beg, int end)
{
   int left, right, temp, loc, flag;
   loc = left = beg;
   right = end;
   flag = 0;
   while(flag != 1)
   {
      while((a[loc] <= a[right]) && (loc!=right))
      right--;
      if(loc==right)
      flag =1;
      else if(a[loc]>a[right])
      {
         temp = a[loc];
         a[loc] = a[right];
         a[right] = temp;
         loc = right;
      }
      if(flag!=1)
      {
         while((a[loc] >= a[left]) && (loc!=left))
         left++;
         if(loc==left)
         flag =1;
         else if(a[loc] < a[left])
         {
            temp = a[loc];
            a[loc] = a[left];
            a[left] = temp;
            loc = left;
         }
      }
   }
   return loc;
}
void quickSort(int a[], int beg, int end)
```

```
{
   int loc;
   if(beg<end)
   {
      loc = partition(a, beg, end);
      quickSort(a, beg, loc-1);
      quickSort(a, loc+1, end);
   }
}
```

Time Complexities of Searching & Sorting Algorithms:

	Best Case	Average Case	Worst Case
Linear Search	O(1)	O(n)	O(n)
Binary Search	O(1)	O(log n)	O(log n)
Bubble Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Merge Sort	O(n log n)	O(n log n)	O(n log n)
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$